

Developer Guide — Command Station

From first project to a fully featured DCC command station

Covers the library on any platform; the TI MSPM0G3507 LaunchPad is the worked example

Generated 2026-09-25 from the repository sources by `documentation/guides/build_pdfs.py`

Table of Contents

1. Introduction	4
1.1 What the Library Does (and Does Not Do)	4
1.2 Platform Support	4
2. DCC Protocol Concepts	5
2.1 Signal Encoding	5
2.2 Packet Format	5
2.3 Address Types	5
2.4 Instruction Types	5
3. Project File Structure	7
4. dcc_user_config.h in Depth	8
4.1 Role and Feature Flags	8
4.2 Sizes and Tuning Constants	8
5. Initialization — command_station.c	10
5.1 The dcc_config_t Struct	10
5.2 Per-Channel Hardware (dcc_output_hw_t)	10
5.3 RailCom Hardware (dcc_railcom_hw_t)	11
5.4 Setup and Main Loop	11
6. ISR Architecture	12
6.1 The 58 μ s Shared Timer	12
6.2 RailCom One-Shot Timer	12
6.3 100 ms Periodic Tick	12
7. Implementing the Drivers	13
8. The Bit Encoder	14
9. The Scheduler	15
9.1 Slots	15
9.2 Priority	15
9.3 Duplicate Combining	15
9.4 Repeat Counts	15
9.5 Auto-Refresh	16
10. Service Mode Programming	17
10.1 ACK Detection	18
11. RailCom	19

12. Callbacks — Where Your Application Lives	20
13. Application API Reference	21
13.1 Main Track (DccApplicationCommandStationMainTrack_)	21
13.2 Packet Builders (DccApplicationCommandStationPacket_load_)	21
13.3 Service Track (DccApplicationCommandStationServiceTrack_)	21
14. Porting to a New MCU	23
15. Unit Testing	24
15.1 Hardware-in-the-Loop	24
16. Troubleshooting	25

1. Introduction

OpenDccCLib is a C library that implements the NMRA DCC (Digital Command Control) protocol for microcontrollers. It is portable across any hardware platform and toolchain, and it allocates no dynamic memory: every buffer is statically sized at compile time, so it fits processors with very little RAM.

This guide walks through building a complete DCC command station, then explains every module so you understand exactly what the code does and how to extend it. Everything stated here is taken from the library source at the time of generation; where a number matters, the header that defines it is named so you can check it.

1.1 What the Library Does (and Does Not Do)

The library handles the protocol: bit encoding with NMRA timing (58 μ s one-bit halves, 116 μ s zero-bit halves), packet construction with the XOR error byte, a multi-slot scheduler with priorities, duplicate combining and auto-refresh, all four NMRA service-mode methods (direct, paged, register, address) with ACK detection, the RailCom cutout and the decoding of RailCom replies, and spec-correct repeat counts for every one-shot packet.

It contains no hardware-specific code. You supply short driver functions that toggle a GPIO, start and stop timers, and read current sense, and the library calls them through function pointers in one `dcc_config_t` struct. It also does not own the H-bridge, the RailCom detector's analog front end, or any track electronics; those are yours.

1.2 Platform Support

Because the library never touches hardware directly, it ports to any processor and toolchain with a C compiler, one periodic timer, and a few GPIOs.

Processor / Board	Example IDE / Toolchain
TI MSPM0G3507 LaunchPad	Code Composer Studio / Theia (the shipped example)
RP2040 / RP2350	Arduino IDE, PlatformIO, Pico SDK
STM32 (F4 and others)	STM32CubeIDE
ESP32	Arduino IDE, PlatformIO
Any ARM Cortex-M	GCC + Makefile

This guide uses the TI MSPM0G3507 as the worked example. All concepts apply equally to any other target; only the two driver files change.

2. DCC Protocol Concepts

2.1 Signal Encoding

The DCC signal is a square wave whose bit value is encoded in the half-period duration. A one-bit has a nominal half-period of 58 μ s (NMRA allows 55–61 μ s). A zero-bit half must be at least 95 μ s; the library sends 116 μ s, two ticks of the shared timer. The command station alternates the track polarity at these intervals. The constants live in `dcc_defines.h`.

Bit	Half-period (library)	NMRA range	Define
One	58 μ s	55–61 μ s	DCC_ONE_BIT_HALF_PERIOD_US
Zero	116 μ s (two ticks)	$\geq 95 \mu$ s, total $\leq 12\,000 \mu$ s	DCC_ZERO_BIT_MAX_TOTAL_DURATION_US; the 100 μ s DCC_ZERO_BIT_HALF_PERIOD_US define is not used by the encoder

2.2 Packet Format

Every packet is: preamble (one-bits) | start bit 0 | address byte | start bit 0 | instruction byte(s) | start bit 0 | XOR byte | end bit 1. The XOR byte is the exclusive-or of all preceding data bytes. The operations-mode preamble length is yours to set with `USER_DEFINED_DCC_PREAMBLE_BITS_OPS`; the library requires at least 14, or 16 when RailCom is compiled in, because the cutout can blank the first preamble bits and a decoder still needs 10 driven ones. Service-mode packets use `DCC_PREAMBLE_BITS_SERVICE = 20`.

2.3 Address Types

Address type	Range	Bytes	Enum
Broadcast	0	1	DCC_ADDRESS_BROADCAST
Short (7-bit)	1–127	1	DCC_ADDRESS_SHORT
Long (14-bit)	128–10239	2, first byte 0xC0–0xE7	DCC_ADDRESS_LONG
Idle	255	1	DCC_ADDRESS_IDLE
Basic accessory	0–511 (board)	2	DCC_ADDRESS_ACCESSORY
Extended accessory	0–2047	2	DCC_ADDRESS_ACCESSORY_EXTENDED

*Accessory builders take the 9-bit **board** address plus an output pair, not a flat 11-bit output number. The library encodes the wire form; do not pre-shift addresses yourself.*

2.4 Instruction Types

Instruction	Mask (<code>dcc_defines.h</code>)	Description
Speed 14/28-step	0x40 reverse / 0x60 forward	Baseline speed and direction
Speed 128-step, analog function	0x20 advanced ops + 0x3F / 0x3D	126 steps plus stop and e-stop; analog function output
Function group 1	0x80	FL, F1–F4
Function group 2a / 2b	0xB0 / 0xA0	F5–F8 / F9–F12

Instruction	Mask (<code>dcc_defines.h</code>)	Description
Feature expansion	0xC0 + sub-instruction	F13–F68, binary state, time and date, system time
CV access, long form	0xE0 (0xEC write, 0xE4 verify, 0xE8 bit)	Operations-mode CV programming (POM)
Consist control	0x12 / 0x13 set, 0x10 clear	Advanced consisting (CV19)

3. Project File Structure

The files in `dcc_lib/` are library code and should not be edited. The files at the top level and in `application_callbacks/` and `application_drivers/` are yours.

<code>command_station/</code>	<- your project folder
<code>command_station.c</code>	<- main entry: config struct, ISRs, main loop
<code>dcc_user_config.h</code>	<- REQUIRED: feature flags and sizes
<code>uart_command_parser.c/h</code>	<- demo UART command-line interface
<code>application_callbacks/</code>	
<code>callbacks_dcc.c/h</code>	<- application event hooks
<code>application_drivers/</code>	
<code>ti_driverlib_dcc_driver.c/h</code>	<- timers, GPIO, current sense (hardware)
<code>ti_driverlib_uart_driver.c/h</code>	<- UART I/O
<code>dcc_lib/ -> ../../../../src/dcc</code>	<- library core (symlink; do not edit)
<code>dcc_config.h/c</code>	- config struct, lifecycle, wiring of every module
<code>dcc_types.h, dcc_defines.h</code>	- types and NMRA constants
<code>dcc_bit_encoder.h/c</code>	- ISR-level bit serialization
<code>dcc_scheduler.h/c</code>	- slots, priority, duplicate combining, auto-refresh
<code>dcc_application_command_station_packet.h/c</code>	- packet builders
<code>dcc_application_command_station_main_track.h/c</code>	- power, send, auto-refresh
<code>dcc_application_command_station_service_track.h/c</code>	- service mode API
<code>dcc_service_mode_*.h/c</code>	- per-mode primitives and task orchestrators
<code>dcc_railcom_cutout.h/c</code>	- cutout timer state machine
<code>dcc_railcom_command_station.h/c</code>	- RailCom receive, datagram assembly
<code>dcc_railcom_utilities.h/c</code>	- 4/8 code words (shared with the decoder role)
<code>dcc_application_main_track.h/c</code>	- legacy pre-refactor API, still compiled; retirement tracked
<code>dcc_application_service_track.h/c</code>	- legacy pre-refactor API, still compiled

4. dcc_user_config.h in Depth

This file decides which role compiles and how much RAM the library reserves. Every constant except `USER_DEFINED_DCC_ACK_DROP_OUT_TOLERANCE_US` (optional, default 116 μ s) is validated in `dcc_types.h` with a `#error`, so a missing one fails the build rather than the run.

4.1 Role and Feature Flags

```
#define DCC_COMPILE_COMMAND_STATION
#define DCC_COMPILE_RAILCOM // cutout + receive; comment out to strip

#define DCC_COMPILE_SERVICE_MODE_DIRECT // primitives
#define DCC_COMPILE_SERVICE_MODE_PAGED
#define DCC_COMPILE_SERVICE_MODE_REGISTER
#define DCC_COMPILE_SERVICE_MODE_ADDRESS

#define DCC_COMPILE_SERVICE_MODE_TASK_DIRECT // orchestrators (read/write/verify sequences)
#define DCC_COMPILE_SERVICE_MODE_TASK_PAGED
#define DCC_COMPILE_SERVICE_MODE_TASK_REGISTER
#define DCC_COMPILE_SERVICE_MODE_TASK_ADDRESS
#define DCC_COMPILE_SERVICE_MODE_TASK_DETECT
```

The four primitive flags (`DIRECT`, `PAGED`, `REGISTER`, `ADDRESS`) require `DCC_COMPILE_COMMAND_STATION`; `dcc_config.h` emits a `#error` otherwise. The `TASK_*` flags are not cross-checked.

`DCC_COMPILE_RAILCOM` is a real compile-time switch: without it every RailCom module, field and function is stripped, and `dcc_types.h` requires a role flag beside it. At runtime, RailCom stays inert until both `main_track.railcom` and `railcom_timer_start` are wired. Service mode is not inert without hardware: with `service_track.current_sense_read` `NULL` the tasks still transmit their sequences and finish with `NO_ACK` (direct) or `ERROR` (the scanning modes).

4.2 Sizes and Tuning Constants

Define	Example value	Meaning
<code>USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT</code>	24	Scheduler slots. One (address, tag) pair per slot, so a loco with speed plus two function groups uses three
<code>DCC_REFRESH_PROMPT_SENDS</code>	3 (library default)	Full-rate sends of a refresh slot after each insert, before it goes cold
<code>DCC_REFRESH_COLD_CYCLES</code>	60 (library default)	Keep-alive interval of a cold refresh slot, in packet cycles (about 0.4 s); 0 disables the tier and restores the flat ring
<code>DCC_REFRESH_COLD_MAX_CYCLE_S</code>	120 (library default)	Longest any refresh slot may go unsent, in packet cycles (about 0.8 s); see 9.5
<code>DCC_REFRESH_CV11_FLOOR</code>	20 (library default)	Smallest decoder CV 11 (0.1 s units, so 2.0 s) the ceiling is meant to stay under; the check is a <code>static_assert</code> in the scheduler test, not in the library. Lower it with the ceiling
<code>USER_DEFINED_DCC_MAX_LOCOS</code>	10	Locomotives tracked by the demo application's loco table
<code>USER_DEFINED_DCC_PREAMBLE_BITS_OPS</code>	18	Operations-mode preamble; ≥ 14 , ≥ 16 with RailCom

Define	Example value	Meaning
USER_DEFINED_ DCC_RAILCOM_B UFFER_DEPTH	4	Ring of decoded RailCom datagrams
USER_DEFINED_ DCC_SERVICE_M ODE_RETRIES	3	Retries before a service-mode operation reports failure
USER_DEFINED_ DCC_ACK_THRES HOLD_MA	60	Current-sense value that counts as an ACK
USER_DEFINED_ DCC_ACK_MIN_D URATION_US	5000	Shortest pulse accepted as an ACK
USER_DEFINED_ DCC_ACK_MAX_D URATION_US	7000	Longest pulse accepted; longer is treated as over-current, not an ACK
USER_DEFINED_ DCC_ACK_DROP OUT_TOLERANCE_ US	116	Gap inside a pulse the ACK counter tolerates

The example values are those of the shipped command-station project; templates/typical/dcc_user_config.h holds a smaller default set. The four DCC_REFRESH_* values are library defaults from dcc_defines.h; define any of them in dcc_user_config.h to override.

5. Initialization — command_station.c

The main file wires the hardware drivers, the library, and your callbacks. The pattern is always the same: define the config struct, initialize hardware, initialize the library, enter the main loop.

5.1 The `dcc_config_t` Struct

All fields are function pointers or timing values. The library calls the pointers when it needs to toggle a pin, start a timer, or notify you.

```
static const dcc_config_t dcc_config = {
    /* common platform drivers: used by the decoder role, not called in a CS-only build */
    .lock_shared_resources = &TI_DccDriver_lock_shared_resources,
    .unlock_shared_resources = &TI_DccDriver_unlock_shared_resources,
    .get_timestamp_usec = &TI_DccDriver_get_timestamp_usec,

    /* shared 58 us timer: clocks both track encoders */
    .shared_timer_start = &TI_DccDriver_shared_timer_start,
    .shared_timer_stop = &TI_DccDriver_shared_timer_stop,

    /* RailCom cutout one-shot timer + per-state periods (0 = spec default) */
    .railcom_timer_start = &TI_DccDriver_railcom_timer_start,
    .railcom_timer_stop = &TI_DccDriver_railcom_timer_stop,
    .railcom_cutout_start_delay_us = 26, /* DELAY -> T_CS = 26 us */
    .railcom_uart_rx_delay_us = 54, /* SETTling -> T_TS1 = 80 us */
    .railcom_ch1_window_us = 97, /* CH1 -> T_TC1 = 177 us */
    .railcom_ch1_ch2_gap_us = 16, /* GAP -> T_TS2 = 193 us */
    .railcom_ch2_window_us = 261, /* CH2 -> T_CE = 454 us */

    .main_track = { .pin_toggle = &TI_DccDriver_main_pin_toggle,
                    .track_power_set = &TI_DccDriver_track_power_set,
                    .railcom = NULL }, /* detector not fitted on the demo */
    .service_track = { .pin_toggle = &TI_DccDriver_svc_pin_toggle,
                      .track_power_set = &TI_DccDriver_svc_track_power_set, /* not called in this release, se
                      .current_sense_read = &TI_DccDriver_current_sense_read },

    /* OPTIONAL application callbacks (NULL = no notification) */
    .on_packet_sent = &CallbacksDcc_on_packet_sent,
};
```

5.2 Per-Channel Hardware (`dcc_output_hw_t`)

Each DCC output channel, main track and service track, has its own set of pointers. With the shared-timer architecture `pin_toggle` does the work; the header still marks the per-channel `timer_start` and `timer_stop` REQUIRED, but `dcc_config.c` substitutes its own shared-timer wrappers and never calls them.

Field	Required?	Description
<code>pin_toggle</code>	REQUIRED	Toggle this channel's DCC output pin. ISR context; keep it to one register write
<code>track_power_set</code>	REQUIRED	Enable or disable the H-bridge for this channel. Called only for the main track; the service-track power hook is not wired in this release, so service-track power_on/off leave it untouched
<code>timer_start / timer_stop</code>	Marked REQUIRED, never called	Per-channel timer; the library uses the shared timer instead

Field	Required?	Description
current_sense_read	Service track	Return milliamps (ADC) or 0 / non-zero (comparator). Used for ACK detection; <code>main_track.current_sense_read</code> is never read
railcom	NULL if no detector	Pointer to a <code>dcc_railcom_hw_t</code> ; only <code>main_track.railcom</code> is read

5.3 RailCom Hardware (`dcc_railcom_hw_t`)

Field	Called by the library at	Your job
<code>begin_railcom_cutout</code>	T_CS	Tri-state the H-bridge
<code>end_railcom_cutout</code>	T_CE	Restore normal drive
<code>uart_rx_enable</code>	T_TS1 and T_TS2	Open the receiver for the Channel 1, then the Channel 2, window
<code>uart_rx_disable</code>	T_TC1 and T_CE	Close it
<code>uart_read</code>	after the cutout, from <code>DccConfig_run()</code>	Pop one received byte; return false when empty
<code>on_railcom_datagram_result</code>	after decoding, main loop	Receive (address, channel, datagram)

Receive contract. The library reads whatever `uart_read` returns after a cutout and cannot tell a window byte from a stray one. Accept receiver bytes only between an enable and the following disable, discard anything outside (the DCC drive waveform seen by the detector between cutouts, or a stale byte from an earlier cutout), and flush at `begin_railcom_cutout`. Gate in the RX interrupt or toggle the peripheral's receiver; the bench firmware in test/compliance does it in the RX interrupt and is verified on the wire.

5.4 Setup and Main Loop

```
int main(void) {
    SYSCFG_DL_init();                /* MCU clocks, GPIO, timers, UART */
    TI_DccDriver_initialize();
    TI_UartDriver_initialize();
    DccConfig_initialize(&dcc_config); /* library ready; track power still OFF */
    UartCommandParser_initialize();

    while (1) {
        DccConfig_run();              /* scheduling, receive drain, callbacks */
        TI_UartDriver_echo_process();
        UartCommandParser_process();
    }
}
```

`DccConfig_run()` must be called continuously. Every application callback fires from inside it, never from an interrupt, so callbacks may use UART or other slow I/O. Do not block in the loop.

6. ISR Architecture

6.1 The 58 μ s Shared Timer

One hardware timer fires every $DCC_ONE_BIT_HALF_PERIOD_US = 58 \mu s$. Its ISR calls `DccConfig_58us_timer_isr()`, which ticks the main-track and service-track bit encoders. A one-bit half is one tick, a zero-bit half is two (116 μs , inside the NMRA range). The same ISR also samples current sense on the service track for ACK detection.

```
void DCC_BIT_TIMER_INST_IRQHandler(void) {
    switch (DL_TimerA_getPendingInterrupt(DCC_BIT_TIMER_INST)) {
        case DL_TIMER_IIDX_ZERO:
            TI_DccDriver_timestamp_tick();
            DccConfig_58us_timer_isr();
            break;
        default: break;
    }
}
```

6.2 RailCom One-Shot Timer

A second timer runs in one-shot mode and drives the five-state cutout machine. The library restarts it for each phase, and its ISR calls `DccConfig_railcom_oneshot_timer_isr()`. See section 11 for the phase table.

The cutout is armed at the packet end bit's **last edge**. The bit encoder's state machine runs one tick ahead of the wire, so the arm is deferred one tick to land on that edge; this is what keeps `T_CS` inside 26–32 μs and leaves the end bit intact. It was measured on the hardware-in-the-loop bench at 27.8–28.0 μs from the decoded end bit.

6.3 100 ms Periodic Tick

A 100 ms timer, `SysTick` in the example, calls `DccConfig_100ms_timer_tick()`. Its only current job is RailCom accessory polling, an accessory packet every 5 s, and only when `main_track.railcom` and `on_accessory_srq` are both set; otherwise it does nothing. Keep calling it so future housekeeping has a home. It is also a convenient place for a heartbeat LED.

7. Implementing the Drivers

The driver files are the only hardware-specific code in a project. This section states the contract each function must meet when porting.

Function	Context	Contract
lock_shared_resources / unlock_shared_resources	any	Disable and re-enable interrupts, or take and release a mutex. Keep the region short. Not called in a CS-only build
get_timestamp_usec	any	Free-running microsecond counter; monotonic, wrap at 2 ³² is fine. Not called in a CS-only build
shared_timer_start(period) / shared_timer_stop	main loop	Start or stop the 58 μ s periodic timer whose ISR calls DccConfig_58us_timer_isr()
railcom_timer_start(period) / railcom_timer_stop	ISR	One-shot timer whose ISR calls DccConfig_railcom_oneshot_timer_isr()
pin_toggle	ISR	One register write that toggles the channel's DCC pin
track_power_set(bool)	main loop	Enable or disable the H-bridge (main track; see 5.2 for the service track)
current_sense_read	ISR (58 μ s)	Return the service-track current. The demo returns 100 or 0 from a digital pin

8. The Bit Encoder

The bit encoder serializes packets one half-bit per tick from ISR context. It is **single-buffered**: it holds one active packet and a `packet_loaded` flag. When the end bit goes out it signals completion; the scheduler loads the next packet from the main loop during the following idle or preamble time. There is no front/back buffer swap.

The encoder walks the packet in order: preamble ones, then for each byte a start bit and eight data bits, then the end bit. At the end bit it also arms the RailCom cutout, as described in 6.2.

9. The Scheduler

9.1 Slots

The scheduler owns a static array of `USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT` slots. Each holds one packet (up to 6 bytes), its address, tag, priority, repeat count, auto-refresh flag and two pacing counters: full-rate sends still owed and packet cycles since the slot was last sent.

`DccApplicationCommandStationMainTrack_send_packet()` inserts a one-shot; `_add_to_auto_refresh()` inserts a refreshed slot. Both reuse an existing slot with the same (address, tag); a `send_packet` that lands on a refresh slot turns it into a one-shot, so re-add the refresh afterwards.

9.2 Priority

One-shots are selected before refresh, highest priority first.

Priority (<code>dcc_priority_enum</code>)	Suggested use (the library assigns none; the demo sends reset, stop and time/date at <code>DCC_PRIORITY_ESTOP</code>)
<code>DCC_PRIORITY_ESTOP</code>	Emergency stop, highest
<code>DCC_PRIORITY_SPEED</code>	Speed and direction
<code>DCC_PRIORITY_FUNCTION</code>	Functions, consist, binary state, analog, time
<code>DCC_PRIORITY_ACCESSORY</code>	Turnouts and signals
<code>DCC_PRIORITY_CV</code>	Operations-mode CV programming
<code>DCC_PRIORITY_IDLE</code>	Idle, lowest

9.3 Duplicate Combining

A new command for the same (address, tag) overwrites the packet in the existing slot instead of taking a second one. Tags (`dcc_tag_enum`) separate command kinds: speed, each function group, accessory, CV, consist, binary state, analog. Rapid throttle changes therefore never fill the scheduler.

9.4 Repeat Counts

A one-shot slot is sent `repeat_count` times; the scheduler decrements after each send and drops the slot at zero. A packet handed over with a count of 0 is therefore **never sent**, and in this release never freed either: it holds its slot until removed by address. Every packet builder sets a spec-correct default from the `DCC_REPEAT_*` table in `dcc_defines.h`, and an application may overwrite the field after the builder returns.

Builders	Default	Basis
CV write, CV bit write (loco POM and accessory)	2	S-9.2.1 2.3.7.3: two identical packets before a decoder modifies a CV; 2.4.3 applies the same to accessory decoders
CV verify, CV bit verify	1	Acts on the first packet
Model date	3	S-9.2.1 2.3.6.2: "at least three times"

Builders	Default	Basis
Model time, system time	1	Once per update
Accessory NOP, accessory stop	1	Maintainer policy
Everything else: speed, functions, e-stop, accessory on/off, consist, binary state, analog, reset, idle	2	S-9.2 section C gives no count; one repeat survives a single lost packet

9.5 Auto-Refresh

Speed and function commands should be repeated so a decoder keeps hearing them. A slot marked for auto-refresh is never dropped until removed, and it is paced in two tiers so a large pool does not slow down the few locomotives being driven:

Value (dcc_ defines.h, overridable)	Default	Effect
DCC_REFRE SH_PROMPT _SENDS	3	Every insert (a new or changed command) is sent this many times at full rate, one per packet cycle, so a single lost packet does not lose the change
DCC_REFRE SH_COLD_C YCLES	60 (about 0.4 s)	After the burst the slot is "cold" and is re-sent once per this many packet cycles as a keep-alive. 0 disables the tier: every refresh slot is sent in turn, as a flat ring
DCC_REFRE SH_COLD_M AX_CYCLES	120 (about 0.8 s)	Ceiling: a slot unsent for this long is "overdue" and goes ahead of every other refresh slot (pending one-shots are always sent first), so a stream of throttle changes cannot hold a locomotive off the track

With `DCC_REFRESH_COLD_CYCLES` above 0, each packet cycle the scheduler ages every refresh slot, then picks, round-robin within each group: an overdue slot; else a slot still in its burst; else a merely-due cold slot. Right after an overdue send a waiting burst goes first, so under overload changes and overdue keep-alives alternate. A changed command therefore reaches the wire within one packet cycle when no one-shot is pending and nothing else is overdue or in its burst, and shares the burst pass fairly with other simultaneous changes. A cycle with nothing due sends an idle packet.

The ceiling exists because of the decoder packet time-out (S-9.2.4 section 4, CV 11): a decoder stops when no packet addressed to it arrives in time, and idle packets do not count. With the worst-case packet (6 bytes, all zero bits, a RailCom cutout, about 15 ms) 120 cycles is 1.8 s, so the documented floor for CV 11 on a layout driven by this library is `DCC_REFRESH_CV11_FLOOR = 20` (2.0 s in the decoder's 0.1 s units) or 0 (off); a `static_assert` in the scheduler test pins the ceiling under that floor. Keeping the keep-alive under a second also keeps idle locomotives visible to RailCom occupancy detectors, which learn addresses from replies to addressed packets.

Two rules from S-9.2 are enforced in the scheduler regardless of pacing: a same-address packet for short addresses 112–127 is never sent within 5 ms of the previous one (an idle spacer is inserted), and an idle packet goes out whenever nothing else is due.

10. Service Mode Programming

Service mode runs on the dedicated programming track. The command station sends specific packet sequences and the decoder answers with a 6 ms current pulse (ACK). The library splits the work into **primitives** (one mode's packet sequences) and **tasks** (read, write and verify orchestration on top of them). Sequence constants are in `dcc_defines.h`: 3 reset packets, 5 command packets, 6 reset packets after, 6 recovery packets. Register verify uses 7 command packets, and register-1 and address-only writes use 10 recovery packets. The ACK window is blanked for the first 2 command packets, an ACK ends the command phase early, and recovery packets are sent only for writes.

Mode	API prefix <code>DccApplicationCommandStationServiceTrack_</code>	Notes
Direct	<code>direct_read_cv</code> , <code>direct_write_cv</code> , <code>direct_read_bit</code> , <code>direct_write_bit</code>	Read is eight bit-verifies, then a byte-verify of the result; bit read is a verify of 1, then of 0 if silent (no ACK at all = <code>NO_ACK</code>); write is write then verify
Paged	<code>paged_read_cv</code> , <code>paged_write_cv</code> , <code>paged_read_bit</code> , <code>paged_write_bit</code>	Page preset then register access; reads scan
Register	<code>register_read_cv</code> , <code>register_write_cv</code> , <code>register_read_bit</code> , <code>register_write_bit</code> , <code>register_verify_value</code> , <code>register_factory_reset</code>	The read, write and verify calls take a <code>dcc_decoder_type_enum</code> (mobile or accessory); <code>register_factory_reset(on_complete)</code> does not
Address	<code>address_read</code> , <code>address_write</code> , <code>address_verify</code> , <code>address_read_bit</code> , <code>address_write_bit</code>	CV1 only, short addresses
Detect	<code>detect_mode(on_detect)</code>	Probes every compiled mode; reports a bitmask of <code>DCC_SERVICE_MODE_SUPPORTED_*</code>

Every task except `detect_mode`, which takes `on_detect(result, supported_modes)`, takes an `on_complete(result, value)` callback; most take an `on_progress(phase, step, estimated_steps)` callback; and each returns `bool`: false means it could not start, because another operation is running, service mode is not active, or an argument is out of range (CV outside 1–1024, bit above 7). Results arrive later, from `DccConfig_run()`. Call `enter_service_mode()` first and `exit_service_mode()` when done.

Result (<code>dcc_service_mode_result_enum</code>)	Meaning
<code>DCC_SERVICE_MODE_SUCCESS</code>	Completed; value holds the byte or bit read
<code>DCC_SERVICE_MODE_NO_ACK</code>	No qualifying current pulse in direct mode: no decoder, wrong mode, or no current sense wired
<code>DCC_SERVICE_MODE_VERIFY_FAIL</code>	Verify after write did not match, or a direct read's final byte verify did not confirm the bits
<code>DCC_SERVICE_MODE_BUSY</code>	A later step's primitive could not start (a concurrent start makes the task call return false instead)
<code>DCC_SERVICE_MODE_ERROR</code>	A paged, register or address read scanned every candidate value without an ACK; no decoder in those modes

Result (dcc_service_mode_result_enum)	Meaning
DCC_SERVICE_MODE_NOT_IN_SERVICE_MODE	Defined but not produced in this release; a task started outside service mode returns false instead

10.1 ACK Detection

The 58 μ s ISR samples `current_sense_read()` on the service track every tick; samples count only inside the ACK scan window, which opens after the second command packet per S-9.2.3. A sample at or above `USER_DEFINED_DCC_ACK_THRESHOLD_MA` extends the current span. A gap longer than `USER_DEFINED_DCC_ACK_DROP_OUT_TOLERANCE_US` (optional, default 116 μ s) closes the span, which is then judged: it is an ACK if it lasted at least `USER_DEFINED_DCC_ACK_MIN_DURATION_US` and no more than `USER_DEFINED_DCC_ACK_MAX_DURATION_US`; a longer span is rejected as over-current. The limits are quantised to 58 μ s samples (5000 μ s is 85 samples, 7000 μ s is 120), and an ACK is recognised at its falling edge. All of this is exercised on the bench through a mock-ACK loopback, including the boundary widths.

11. RailCom

After each main-track packet the command station opens a cutout by tri-stating the H-bridge, and a decoder answers with a current signal that a detector converts to 250 kbaud UART data. Channel 1 carries two bytes (address broadcast); Channel 2 up to six (CV read-back, status). The library runs the timing, tells your hardware when to tri-state and when to listen, decodes the 4/8 code words and assembles datagrams; you receive finished datagrams tagged with the loco address of the packet whose cutout carried them (cutouts after accessory, broadcast or idle packets are tagged 0).

State	Duration (default)	Boundary	Hook called
DELAY	26 μ s	T_CS = 26 μ s after the end bit	begin_railcom_cutout
SETTLING	54 μ s	T_TS1 = 80 μ s	uart_rx_enable
CH1	97 μ s	T_TC1 = 177 μ s	uart_rx_disable
GAP	16 μ s	T_TS2 = 193 μ s	uart_rx_enable
CH2	261 μ s	T_CE = 454 μ s	uart_rx_disable, end_railcom_cutout

The five durations are configurable in `dcc_config_t` (0 selects the default from `dcc_defines.h`) and at runtime with `DccConfig_set_railcom_cutout_timing()`. `DccConfig_cancel_railcom_cutout()` aborts an in-progress cutout and restores the bridge; `DccConfig_railcom_cutout_is_active()` reports state.

Decoded datagrams carry a 4-bit id (`DCC_RAILCOM_ID_*`: POM 0, ADR1 1, ADR2 2, EXT 3, DYN 7, XPOM 8–11, CV auto 12, time 14, logon enable 15) and up to six data bytes. ACK (0x0F or 0xF0) and NACK (0x3C) are code words, not datagrams; BUSY (0xE1) is defined but decodes as invalid on the receive side; a datagram followed by ACK padding is kept. The 4/8 table follows the S-9.3.2 draft's Table 2 (the released 2012 table is identical for the 64 data words).

Known limitation. Received bytes are split into channels by count: the first two are treated as Channel 1, the rest as Channel 2. A reply that contains only Channel 2 data (a decoder with the Channel 1 broadcast disabled through CV 28) is reported as a Channel 1 datagram. Tracked as an open issue; the bench suite keeps a deliberately failing case for it.

12. Callbacks — Where Your Application Lives

Command-station callbacks live in `dcc_config_t` and fire from `DccConfig_run()`.

Callback	Fires when
<code>on_packet_sent(const dcc_packet_t*)</code>	The scheduler has dispatched a packet to the encoder (transmit start), on either track, service-mode resets included; main-track idle packets do not fire it. The bench firmware uses it to pulse a scope trigger
<code>on_accessory_srq(address, is_extended)</code>	Reserved: nothing calls it in this release. Setting it together with <code>main_track.railcom</code> enables the 100 ms accessory polling in 6.3
<code>on_railcom_datagram_result(address, channel, datagram)</code>	In <code>dcc_railcom_hw_t</code> ; a datagram was decoded
<code>on_complete / on_progress / on_detect</code>	Per service-mode call; see section 10

13. Application API Reference

All functions are in the three `dcc_application_command_station_*.h` headers. Builders fill a `dcc_packet_t` and return false on a bad argument (the argument-free idle, reset, e-stop and system-time builders return void); they do not send. Sending is a main-track call.

13.1 Main Track (`DccApplicationCommandStationMainTrack_`)

Function	Description
<code>power_on()</code> / <code>power_off()</code>	Track power and DCC generation on the main track
<code>send_packet(packet, address, tag, priority)</code>	One-shot; sent <code>repeat_count</code> times
<code>add_to_auto_refresh(packet, address, tag, priority)</code>	Refreshed slot; replaces an existing (address, tag)
<code>remove_from_auto_refresh(address)</code>	Drop every active slot for an address, refresh slots and pending one-shots alike
<code>remove_all_auto_refresh()</code>	Idle-only stream

13.2 Packet Builders (`DccApplicationCommandStationPacket_load_`)

Builder	Description
<code>idle, reset, estop_all(isPanic)</code>	Idle, broadcast reset, broadcast stop (panic = e-stop, else controlled stop)
<code>speed_128(addr, type, speed, dir), speed_28(...), speed_14(..., headlight)</code>	Speed and direction
<code>func_group_1, func_group_2a, func_group_2b</code>	FL/F1–F4, F5–F8, F9–F12
<code>func_f13_f20 ... func_f61_f68</code>	Function expansion groups
<code>accessory_basic(board, pair, activate), accessory_extended(addr, aspect), accessory_nop(addr, is_extended)</code>	Turnouts, signals, RailCom polling
<code>accessory_basic_stop, accessory_extended_stop</code>	Stop (collect a RailCom service request)
<code>accessory_basic_cv_write/verify/bit, accessory_extended_cv_write/verify/bit</code>	Accessory operations-mode CV access
<code>cv_write_pom, cv_verify_pom, cv_bit_pom</code>	Loco operations-mode CV access
<code>consist_set(addr, type, consist, normal), consist_clear</code>	Advanced consist
<code>binary_state_short, binary_state_long, analog_function</code>	Feature expansion
<code>system_time(ms), model_time(...), model_date(day, month, year)</code>	Broadcast time and date

13.3 Service Track (`DccApplicationCommandStationServiceTrack_`)

`power_on/off`, `enter_service_mode`, `exit_service_mode`, `is_service_mode_active`, and the per-mode tasks listed in section 10. `power_on/off` start and stop the service-track encoder and timer only; they do

not call `track_power_set` in this release. `exit_service_mode` is ignored by the service-mode core while an operation is running, although the encoder and timer are still stopped.

14. Porting to a New MCU

1. Copy the `command_station` example project.
2. Rewrite `ti_driverlib_dcc_driver.c/h` for your timers, GPIO and current sense, meeting the contracts in section 7.
3. Rewrite or drop `ti_driverlib_uart_driver.c/h` (the CLI is optional).
4. Set the sizes in `dcc_user_config.h` for your RAM.
5. Wire your ISRs to `DccConfig_58us_timer_isr()`, `DccConfig_railcom_oneshot_timer_isr()` and `DccConfig_100ms_timer_tick()`.
6. Build, then verify on a logic analyzer: 58 μ s one halves, 116 μ s zero halves, and a cutout that starts 26–32 μ s after the end bit.

Minimum hardware: one periodic timer at 58 μ s and one GPIO per track channel (a microsecond timestamp is needed only by the decoder role). Optional: a one-shot timer for RailCom, an ADC or comparator for ACK sensing, a 250 kbaud UART fed by a RailCom detector.

15. Unit Testing

Tests are GoogleTest C++ files beside the sources, `src/dcc/*_Test.cxx`, one binary per file. They run against mocked drivers, so a green host suite says the protocol logic is right, not that a board is wired right; the hardware-in-the-loop suite below covers the wire.

```
cd test
make          # configures CMake, builds, runs every binary serially, writes test/coverage.html
```

At generation time: 29 test binaries, 1244 tests, 0 failures, 0 warnings; line coverage 99.7 %, function coverage 100 %, branch coverage 98.2 % (gcovr). The build also compiles six single-role configurations so a missing `DCC_COMPILE_*` guard fails as a compile or link error.

Test file	What it tests
<code>dcc_bit_encoder_Test</code>	Bit timing, preamble, framing, cutout armed at the end bit's last edge
<code>dcc_scheduler_Test</code>	Priority, duplicate combining, refresh pacing (burst, keep-alive, ceiling, fairness, flat-ring mode, CV 11 floor), 5 ms spacing, repeat counts untouched and overridden
<code>dcc_application_command_station_packet_Test</code>	Byte-exact vectors for every builder plus its repeat default
<code>dcc_application_command_station_main_track_Test</code> , <code>..._service_track_Test</code>	Application API
<code>dcc_application_main_track_api_Test</code> , <code>dcc_application_service_track_api_Test</code>	The legacy pre-refactor API modules
<code>dcc_service_mode_{direct,paged,register,address,common}_Test</code>	Per-mode primitives, ACK detection, failed-start handling
<code>dcc_service_mode_task_{direct,paged,register,address,detect}_Test</code>	Read/write/verify orchestration and detect
<code>dcc_railcom_cutout_Test</code> , <code>dcc_railcom_command_station_Test</code> , <code>dcc_railcom_utilities_Test</code>	Cutout state machine, receive and datagram assembly, 4/8 code words
<code>dcc_config_Test</code> , <code>dcc_failsafe_Test</code> , decoder-side tests	Wiring, lifecycle, ISR dispatch; the decoder role

15.1 Hardware-in-the-Loop

`test/compliance/` holds a Saleae logic-analyzer bench that drives the shipped firmware over UART and checks the wire against the NMRA standards: S-9.1 timing, S-9.2 baseline packets, S-9.2.1 packet bytes and repeat counts, S-9.2.3 service mode with a mock-ACK loopback, S-9.3.2 cutout timing and sub-windows plus the receive path through a mock-decoder loopback, and the scheduler. A preflight script proves every probe and jumper before a run. See `test/compliance/command_station/HIL_SETUP.md`.

16. Troubleshooting

Symptom	Likely cause and fix
No DCC on the scope	Track power is off; type POWER ON. Check pin_toggle wiring
Bit timing wrong	Shared timer period is not 58 μ s; check the MCU clock tree
Decoder does not respond	H-bridge wiring, or the address in SPEED does not match the decoder
A one-shot command never appears	Its repeat_count is 0; the builders set defaults, an override to 0 means never send
SVC RESULT: NO ACK	Direct mode: no decoder on the programming track, current_sense_read not wired, or threshold too high
SVC RESULT: ERROR	Paged, register or address read scanned every value without an ACK: no decoder, or threshold too high
ERR: service mode operation failed to start	Another service-mode operation is running, or SVC ENTER was not issued; wait for the result or enter service mode
ERR: scheduler full	Raise USER_DEFINED_DCC_SCHEDULER_SLOT_COUNT
#error about a USER_DEFINED_*	The constant is missing from dcc_user_config.h, or the file is not on the include path
RailCom datagrams missing or mislabeled	Receiver not gated to the windows (see 5.3), stale bytes not flushed, or the detector's UART is not 250 kbaud 8N1

Library source: github.com/JimKueneman/OpenDccCLib. NMRA DCC standards: www.nmra.org/dcc.